An open format for reconstructible reporting

For evaluators who use AI on interviews, reports and other text, and for the people who commission them.

Reconstructible reporting means handing over, with your report, the data and enough detail for someone else to rebuild your analysis and run it again. In particular, any frontier AI should be able to do it out of the box without further information or instructions.

We think it should be easy for evaluators to show that they used AI in ways somebody else can check. Rubicon writes its reports in this format, but you do not need Rubicon to report reconstructibly. We have published the format so that anyone can use it: version 0.3.

Why

Ask a model to "Read these forty interviews and tell me whether the programme worked" and it has to decide everything itself. Ask it to "Mark every passage where a respondent describes an increase in income; count how many interviews contain at least one; judge that count against this rubric" and it has much less to decide. Breaking a fuzzy request into simpler ones makes the work auditable and puts every judgement against an explicit rubric. It can also help clarify what we meant in the first place. We call the set of steps a **workflow**.

With explicit steps you can also see what is missing. A step that finds nothing has to show what was searched and how much was read. A proportion has to state its base, including the texts that said nothing.

The bundle

With your report, hand over a **bundle**: the sources; the workflow as a text file, with every setting it ran with, including which AI model did each step; what every step produced, such as each coding table with its quotations, each codebook and each count with the documents behind it; the rubrics; every call made to a model; and how to read all of it. The bundle is machine-readable, so that a competent reader with time, or an AI in minutes, can rebuild the workflow and run it again without any particular software.

The format

In our format, every workflow is built from six general pieces:

- **sample** chooses documents;
- **code** marks passages and records what each says, in whatever columns the question needs;
- **group** finds the themes behind open codes and writes them up as a codebook, each theme with what counts and what does not;
- **tabulate** counts, by software alone, always out of a stated base that includes the documents that said nothing;
- **judge** applies the evaluator's rubric to the counts and passages;
- **write** gives the answer, in which every number comes from a count and every claim has a citation to its passages.

Each piece has a few settings, each with a default. Every setting used in a run is recorded. The format is published, now at version 0.3. A published version never changes; a correction goes out as a new version alongside it. Use it, fork it or ignore it. We do not need everyone to agree on *one* format, because a bundle that includes its own description can be read by somebody who has never seen the format.

What a rerun proves

A rerun with AI will not always give exactly the same results. A step that branches on a threshold such as "more than half", for example, can go the other way when the count is close to 50%. We claim that the process can be rebuilt and run again, giving findings in the same ballpark that can be compared. In the terms of Goodman, Fanelli and Ioannidis (2016), this is *methods* reproducibility, in a form that runs. *Results* reproducibility is "nice to have". The smaller and more explicit the steps, particularly early on, the better the chance that a rerun produces the same results.

What the format does not do

The format does not cover how to *design* the workflow. In Rubicon you design it in dialogue with the assistant. There is no algorithm for the right way to answer an evaluation question. Some clients want the workflow fixed early; others are happy for you to experiment. All the format requires is that, once you have run the workflow, you can hand over how it was done.

Our format is for processing texts and narratives, though the same idea could work for workflows that include statistical analysis.

Existing reporting standards

Evaluation has the CHESS checklist (Montrosse-Moorhead and Griffith, 2017); qualitative research has COREQ and SRQR. These set out how to report what you did. With our format you also report it in a form that runs. For our survey, see Is there already a standard?

More ideas

What makes a question well defined

One that two people with graduate-level social science skills would answer much the same way from the same material. In the problem-solving literature a problem is ill-defined when its statement leaves parts open (Reitman, 1964), and well and ill defined are the ends of a continuum (Simon, 1973). Most evaluators mix and adapt methods to the case in hand (Aston and Apgar, 2022; Apgar and others, 2024). A format like this does not prevent that, provided the mix is written down.

Pre-registering the workflow, or not, or half-way

A workflow can be registered in advance with a commissioner, with a version, a date and a named person. Register nothing and you can still hand over a workflow somebody else can read *and run*. Register the sampling and the coding and you have committed to the descriptive half. Register the rubric too and you have said what counts as good before you know what the material says. (Some people love this, some people hate it.) You can still iterate. Every workflow version is dated and never rewritten, and the version in force is recorded for each run, so a reader can see whether a threshold was moved before or after the results came in.

You can also write a workflow up afterwards, describing only the arrangement you settled on after experimenting. Somebody else can still read, check and rerun it.

Rubrics somebody can apply

With a rubric, the evaluator turns a count or a set of passages into a verdict: is this good, or good enough? Each level needs a description detailed enough that somebody could hold it against real material and say whether the material meets it. If a level is only vaguely described, the reasoning has to be rebuilt case by case. Where one judgement depends on several criteria, the rubric should state how they combine, so that whoever reads the table does not have to decide. The format does not include thresholds such as "overwhelming means eight out of ten". The evaluator sets them; the format makes the setting visible. Sometimes the people involved do not want criteria settled in advance, because the material may turn out richer than whoever wrote the question expected. So a workflow can stop at a table or a piece of prose without reaching a verdict.

Room for participation and reflection

In many approaches, Outcome Harvesting among them, participation and the analyst's own reflection are the most important part of the work. In a format like this, a phase of a workflow can be marked as outside any algorithm: the work pauses while people meet or think, then continues with what they decided, or starts again. One workflow can then be participatory where it needs to be and rule-bound and checkable elsewhere. The reader can see which parts are which.

Why not just an appendix of prompts?

From somebody's prompts you can learn what they meant to do. You cannot run them, and you cannot see from them what the run actually did. A prompt also only makes sense in the conversation around it. A careful reader of an appendix will still miss three common faults, because each is in the results and the appendix shows only the plan: a quotation that appears nowhere in the source; a proportion calculated without the documents that said nothing; a threshold set after the numbers came in. A commissioner who receives forty evaluations a year will not read forty folders of prompts, but a machine can check forty bundles against a published format.

Validating the workflow

- Generate two sets of synthetic documents, one that should score well and one that should score badly, and run the workflow over both before the real material.
- Develop the workflow on a subset and run the settled version over the rest.
- Run one instruction twice over the same ten sources to see the run-to-run variation, and two wordings to see how sensitive the instruction is.

See also: Pilots, not passengers — reconstructible reporting in evaluation; Principles for reconstructible analysis in the age of AI.

This page was drafted with the assistance of Claude, an AI model from Anthropic; the arguments and judgements are our own.